Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Informatyki

Praca dyplomowa inżynierska

na kierunku Informatyka
w specjalności Sztuczna Inteligencja

Projekt inteligentnej bramy (Gateway) oraz narzędzia CLI do semantycznej deduplikacji danych w systemach Cloud Storage

Kacper Górski
Numer albumu 331379

promotor
dr. inż. Łukasz Lepak

WARSZAWA 2026

Projekt inteligentnej bramy (Gateway) oraz narzędzia CLI do semantycznej deduplikacji danych w systemach Cloud Storage

Streszczenie. Chmurowe magazyny obiektowe, takie jak Amazon S3, pozwalają na niemal nieograniczoną skalowalność przechowywania danych. W systemach, do których trafia duża liczba plików graficznych, np. w serwisach e-commerce, mediach społecznościowych czy archiwach zdjęć, model rozliczeniowy Pay-As-You-Go sprawia jednak, że utrzymywanie redundantnych danych generuje wysokie koszty. Ten sam obraz jest często przesyłany wielokrotnie w różnych rozdzielczościach, formatach (np. PNG i silnie skompresowany JPEG) lub z drobnymi artefaktami kompresji. Klasyczne podejście do weryfikacji tożsamości danych, oparte na funkcjach skrótu (takich jak MD5 czy SHA-256), okazuje się w takiej sytuacji niewystarczające. Ze względu na tzw. efekt lawinowy zmiana choćby jednego bajtu pliku daje zupełnie inny skrót, przez co obrazy niemal identyczne wizualnie nie są rozpoznawane jako duplikaty, a przestrzeń dyskowa jest marnowana.

W pracy przedstawiono koncepcję oraz implementację inteligentnej bramy deduplikacyjnej, która wykorzystuje modele sztucznej inteligencji do analizy semantycznej przesyłanych obrazów. Zamiast porównywać zawartość bajtową, system przekształca każdy obraz w wielowymiarowy wektor (ang. *embedding*) za pomocą modelu SSCD, przeznaczonego do wykrywania kopii obrazów, a podobieństwo wektorów wyznacza miarą kosinusową (ang. *cosine similarity*). Gdy zostanie wykryty obraz o wysokim stopniu podobieństwa do już przechowywanego, mechanizm porównuje jakość obu wersji na podstawie metryk bezreferencyjnych, m.in. BRISQUE oraz wariancji laplasjanu jako miary ostrości. Dzięki temu duplikaty gorszej jakości są odrzucane, a słabsze wersje obrazów mogą zostać zastąpione wersjami o lepszej jakości.

Zaimplementowana architektura hybrydowa składa się z autorskiego narzędzia CLI napisanego w języku Go, które pośredniczy w operacjach zapisu, oraz połączonego z nim lokalnego mikroserwisu w języku Python, w którym uruchomiony jest model wektorowy. Logika przetwarzania działa w środowisku lokalnym i jest bezpośrednio zintegrowana z usługami chmury AWS. W celu zwiększenia wydajności proces podzielono na dwa etapy. W pierwszym dokładne duplikaty są wykrywane z użyciem bazy klucz-wartość Amazon DynamoDB, co pozwala odrzucić powtarzające się pliki w czasie $O(1)$. W drugim nowe dane poddawane są właściwej analizie semantycznej. Reprezentacje wektorowe przechowywane są w bazie PostgreSQL z rozszerzeniem pgvector, umożliwiającym wydajne wyszukiwanie najbliższych sąsiadów, natomiast zdeduplikowane pliki trafiają do standardowych zasobników Amazon S3. Część badawcza pracy osadzona jest w metodyce FinOps i ma na celu określenie, przy jakiej skali i charakterystyce danych wdrożenie takiego mechanizmu przynosi wymierne korzyści finansowe.

Słowa kluczowe: Cloud Storage, Amazon S3, deduplikacja semantyczna, deduplikacja obrazów, embeddingi, SSCD, ocena jakości obrazu, Amazon DynamoDB, pgvector, FinOps

Design of an intelligent gateway and a CLI tool for semantic data deduplication in cloud storage systems

Abstract. Cloud object storage services such as Amazon S3 offer almost unlimited scalability. In systems that receive large numbers of image files, e.g. e-commerce platforms, social media or photo archives, the Pay-As-You-Go billing model makes storing redundant data expensive. The same image is often uploaded many times in different resolutions, formats (e.g. PNG and heavily compressed JPEG) or with minor compression artifacts. The classic approach to verifying data identity, based on hash functions such as MD5 or SHA-256, is not sufficient in this case. Because of the avalanche effect, changing even a single byte of a file produces a completely different hash, so visually near-identical images are not recognised as duplicates and storage space is wasted.

This thesis presents the design and implementation of an intelligent deduplication gateway that uses artificial intelligence models to perform semantic analysis of uploaded images. Instead of comparing raw bytes, the system converts each image into a high-dimensional vector (embedding) using the SSCD copy detection model and measures similarity between vectors with cosine similarity. When an image highly similar to an already stored one is detected, the mechanism compares the quality of both versions using no-reference metrics, including BRISQUE and the variance of the Laplacian as a sharpness measure. As a result, lower-quality duplicates are rejected and weaker versions of images can be replaced with better ones.

The implemented hybrid architecture consists of a custom CLI tool written in Go, which mediates write operations, and a connected local Python microservice that runs the embedding model. The processing logic runs locally and is directly integrated with AWS cloud services. To improve performance, the process is divided into two stages. First, exact duplicates are detected using the Amazon DynamoDB key-value database, which allows repeated files to be rejected in $O(1)$ time. Second, new data undergoes the actual semantic analysis. Vector representations are stored in PostgreSQL with the pgvector extension, which enables efficient nearest-neighbour search, while deduplicated files are stored in standard Amazon S3 buckets. The research part of the thesis follows the FinOps methodology and aims to determine at what scale and for which data characteristics deploying such a mechanism brings measurable financial benefits.

Keywords: Cloud Storage, Amazon S3, semantic deduplication, image deduplication, embeddings, SSCD, image quality assessment, Amazon DynamoDB, pgvector, FinOps

Spis treści

1. Praefatio 7

2. Analiza dziedziny i przegląd technologii 9

2.1. Platforma Amazon Web Services 9

2.2. Przechowywanie danych w chmurze – Amazon S3 9

2.2.1. Architektura danych: Zasobniki i Obiekty 10

2.2.2. Zarządzanie metadanymi 10

2.2.3. Model rozliczeniowy i optymalizacja kosztów (FinOps) 10

2.3. Amazon DynamoDB jako szybki indeks duplikatów 11

2.4. Klasyczna deduplikacja oparta na funkcjach skrótu 12

2.4.1. Algorytm MD5 (*Message-Digest Algorithm 5*) 12

2.4.2. Algorytm SHA-256 (*Secure Hash Algorithm 2*) 12

2.4.3. Porcjowanie zależne od treści (CDC) 13

2.4.4. Wnioski dla deduplikacji obrazów 13

2.5. Reprezentacja semantyczna i wektoryzacja obrazów (Embeddings) 13

2.5.1. Konwolucyjne sieci neuronowe i przestrzeń wielowymiarowa 14

2.5.2. Matematyczny model uczenia samonadzorowanego (SimCLR i SSCD) 14

2.5.3. Porównywanie obrazów i miara cosinusowa 15

2.6. PostgreSQL z rozszerzeniem pgvector 16

2.7. Komunikacja między usługami: gRPC i Protocol Buffers 16

3. Architektura systemu deduplikacyjnego 18

3.1. Założenia projektowe i wymagania 18

3.2. Podział na komponenty i warstwę danych 18

3.2.1. Dobór technologii: Go oraz Python 19

3.2.2. Komunikacja za pomocą gRPC 20

3.3. Logiczny potok deduplikacyjny 21

3.3.1. Wstępne wyszukiwanie z wykorzystaniem CDC 21

3.3.2. Weryfikacja pełnego skrótu i warunki kontynuacji 22

3.4. Integracja z modułem analitycznym i obsługa wyników 23

3.4.1. Przesłanie i buforowanie obrazu 23

3.4.2. Decyzja AI Workera 24

3.4.3. Kolejność zapisu i wycofanie poprzedniej wersji 25

3.5. Miejsce oceny jakości w potoku 25

3.6. Współbieżność i zakres gwarancji spójności 26

3.6.1. Limity i anulowanie pracy 26

3.6.2. Transakcja decyzji i operacje między magazynami 26

4. Moduł sztucznej inteligencji i analiza semantyczna obrazów 28

4.1. Ekstrakcja cech i model SSCD (Self-Supervised Descriptor) 28

4.2. Reprezentacja wektorowa i przeszukiwanie przestrzeni wielowymiarowej . . 28

4.3. Obiektywna ocena jakości obrazu (InfoGain) i logika decyzyjna 28

4.4. Ewaluacja i strojenie progów podobieństwa (Threshold Tuning) 28

5. Ewaluacja, strojenie progów i analiza efektywności (FinOps) 29
5.1. Metodologia eksperymentu i przygotowanie danych 29
5.2. Strojenie progów podobieństwa (Threshold Tuning) 29
5.3. Wydajność potoku deduplikacyjnego (Pipeline Performance) 29
5.4. Analiza finansowa i optymalizacja kosztów chmurowych (FinOps) 29
6. Summatio 30
Bibliografia 33
Wykaz symboli i skrótów 37
Spis rysunków 37
Spis tabel 37
Spis załączników 37

1. Praefatio

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum. [1]



Rysunek 1.1. Tradycyjne godło Politechniki Warszawskiej

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris. Reference to image 1.1.

- Item 1:
  - item 1.1;
  - item 1.2;
- Item 2;
- Item 3.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante.

Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

1. Item 1:
   a) item 1.1;
   b) item 1.2:
      i. item 1.2.1;
      ii. item 1.2.2;
   c) item 1.3;
2. Item 2;
3. Item 3.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula. Lorem ipsum dolor sit amet[1], consectetur adipiscing elit.

Tabela 1.1. Przykładowa tabela.

Kolumna 1	Kolumna 2	Liczba
cell1	cell2	60
cell4	cell5	43
cell7	cell8	20,45
	Suma[2]	123,45

Lorem ipsum dolor sit amet.

[1] Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

[2] Table footnote.

2. Analiza dziedziny i przegląd technologii

Przed rozpoczęciem pracy należy wprowadzić koncepcje i technologie wykorzystane do budowy inteligentnej bramy deduplikacyjnej. Opisano czym właściwie jest chmura publiczna Amazon Web Services, specyfikę obiektowych magazynów danych oraz baz dokumentowych na podstawie usług Amazon S3 oraz Amazon DynamoDB. Skonfrontowano także tradycyjne, oparte na funkcjach skrótu metody weryfikacji powielonych danych z podejściem semantycznym, które bazuje na wektorowych modelach wizyjnych.

2.1. Platforma Amazon Web Services



Rysunek 2.1. Logo Amazon Web Services. Źródło: Wikimedia Commons [2].

Amazon Web Services (AWS) stanowi zbiór usług chmurowych udostępnianych przez firmę Amazon, obejmujący ponad dwieście produktów przeznaczonych do przechowywania, przetwarzania oraz analizy danych [3]. Zasoby przydzielane są na żądanie, a opłaty naliczane według rzeczywistego zużycia – bez obowiązku wcześniejszej rezerwacji infrastruktury [4]. Platforma działa w ramach regionów geograficznych. Każdy z nich obejmuje co najmniej dwie odseparowane strefy dostępności, połączone siecią o niskich opóźnieniach [5].

W opracowywanej bramie deduplikacyjnej wykorzystano trzy usługi tej platformy. Amazon S3 służy jako magazyn obiektowy dla plików graficznych wraz z metadanymi. Amazon DynamoDB pełni funkcję indeksu dokładnych duplikatów, przeszukiwanego za pomocą operacji `Query` i `GetItem` [6], [7]. Komponent obliczeniowy pracuje lokalnie, lecz współpracuje z usługami AWS za pośrednictwem oficjalnych SDK [8] – dzięki temu całe rozwiązanie można przenieść do chmury bez modyfikacji logiki biznesowej.

Rozliczenia w AWS opierają się na modelu Pay-As-You-Go: koszt usługi zależy od zajętej przestrzeni dyskowej, liczby wykonanych operacji API oraz objętości transferu danych [4], [9]. Zależność ta stanowi bezpośrednią motywację dla deduplikacji – odrzucenie każdej kopii obrazu eliminuje koszt przechowywania i zapisu jej danych binarnych, nie wyklucza jednak osobnego zapisu metadanych.

2.2. Przechowywanie danych w chmurze – Amazon S3



Rysunek 2.2. Ikona usługi Amazon S3. Źródło: AWS Architecture Icons [10].

Amazon Simple Storage Service (S3) to usługa typu *Object Storage*, przeznaczona do przechowywania dużych zbiorów danych w postaci binarnej (BLOB) [11], [12]. W odróżnieniu od tradycyjnych systemów plików o strukturze drzewiastej, magazyny obiektowe przechowują dane w płaskiej przestrzeni, a widoczne w interfejsie użytkownika foldery są jedynie logicznym grupowaniem. Takie podejście ułatwia rozpraszanie danych w chmurze, zapewniając wysoką dostępność i łatwość skalowania.

2.2.1. Architektura danych: Zasobniki i Obiekty

Struktura S3 opiera się na dwóch podstawowych pojęciach [11], [12]:

- **Zasobniki (ang. *Buckets*)** – Kontenery najwyższego poziomu. Domyślnie nazwa każdego zasobnika musi być unikalna w skali globalnej (od 2026 roku można też tworzyć zasobniki we własnej, regionalnej przestrzeni nazw konta [13]), choć fizycznie przypisuje się go do konkretnego regionu AWS (np. `us-east-1` znajdujący się w Północnej Wirginii w USA). Na poziomie zasobnika konfiguruje się m.in. polityki dostępu IAM oraz reguły retencji danych.
- **Obiekty (ang. *Objects*)** – Właściwe pliki przechowywane w zasobnikach. Na obiekt składają się dane binarne (do 50 TB na pojedynczy plik [14]), metadane oraz unikalny klucz (ang. *Key*), który pełni rolę identyfikatora w ramach zasobnika.

2.2.2. Zarządzanie metadanymi

Metadane w S3, zapisywane jako pary klucz-wartość, dostarczają dodatkowych informacji o obiektach i dzielą się na dwie grupy [11], [12]:

- **Systemowe** – Generowane przez AWS (np. rozmiar pliku, data modyfikacji, znacznik *ETag*). W przypadku obiektów przesłanych pojedynczym żądaniem `PUT`, bez szyfrowania kluczem SSE-KMS lub SSE-C, *ETag* jest skrótem MD5 zawartości pliku [15].
- **Użytkownika** – Niestandardowe parametry nadawane przez aplikację podczas tworzenia obiektu, przesyłane w nagłówkach HTTP z prefiksem `x-amz-meta-`.

W bramie deduplikacyjnej opis pliku przechowywany jest jako osobny obiekt JSON. Otrzymuje on klucz `metadata/<file_name>.json`, a dane obrazu zapisywane są niezależnie pod `payloads/<file_name>`. Opis JSON należy odróżnić od metadanych użytkownika w nagłówkach `x-amz-meta-`, które są atrybutami dołączonymi do obiektu S3 [16]. JSON przesyłany jest jako treść obiektu metodą `PutObject`, która zapisuje dane pod wskazanym kluczem w zasobniku [17]. Aktualizacja tego opisu nie wymaga kopiowania obrazu na niego samego. Kolejność zapisu metadanych i obrazu przedstawiono w podrozdziale 3.4.

2.2.3. Model rozliczeniowy i optymalizacja kosztów (FinOps)

Użycie usług takich jak S3 wiąże się z modelem płatności *Pay-As-You-Go*, w którym koszty zależą od faktycznego zużycia zasobów. Składają się na nie trzy główne elementy [11], [12]:

1. **Wolumen danych** – Miesięczna opłata za fizycznie zajętą przestrzeń.
2. **Transfer (Data Egress)** – Koszt przesyłania danych poza infrastrukturę AWS.

3. **Operacje API** – Opłaty za żądania sieciowe, przy czym operacje modyfikujące stan (`PUT`, `COPY`, `POST`) są znacznie droższe niż operacje odczytu (`GET`).

Wdrożenie inteligentnej bramy deduplikacyjnej ma na celu redukcję tych wskaźników. Odrzucanie powielonych obrazów ogranicza zapis danych binarnych w S3, nie eliminuje jednak wszystkich operacji zapisu. Po decyzji deduplikacji nadal zapisywany jest osobny obiekt JSON z metadanymi; służy do tego `PutObject`, czyli operacja dodania obiektu do zasobnika [17]. W analizie kosztów należy więc uwzględnić zarówno pominięty zapis obrazu, jak i zapis JSON-a.

2.3. Amazon DynamoDB jako szybki indeks duplikatów



Rysunek 2.3. Ikona usługi Amazon DynamoDB. Źródło: AWS Architecture Icons [10].

Amazon DynamoDB to zarządzana, rozproszona baza danych NoSQL typu klucz-wartość, zapewniająca stałą, niską latencję dostępu przy dużych wolumenach zapisu i odczytu [18], [19]. Rekord identyfikowany jest przez klucz główny (ang. *primary key*), składający się z klucza partycji (ang. *partition key*) albo pary: klucz partycji i klucz sortowania (ang. *sort key*). Przy kluczu złożonym dopiero ta para jednoznacznie identyfikuje rekord [20].

W projektowanej bramie deduplikacyjnej DynamoDB pełni funkcję indeksu dokładnych duplikatów. Do wstępnego wyszukiwania wykorzystywany jest skrót SHA-256 pierwszego fragmentu wyznaczonego przez CDC, a nie skróty wszystkich fragmentów pliku. Zastosowana biblioteka `restic/chunker` wyznacza granice na podstawie kroczącego odcisku Rabina [21]. Skróty c i h, oznaczające odpowiednio skrót początkowego fragmentu i całego pliku, tworzą rekord `PK = CHUNK#<c>`, `SK = FULL#<h>`. Osobny rekord `PK = FILE#<file_name>`, `SK = META` przechowuje metadane potrzebne do zapisu i podmiany obiektu; nie stanowi dodatkowego etapu detekcji duplikatu.

Wyszukiwanie rozpoczyna się od `Query` dla klucza partycji `CHUNK#<c>` z parametrem `Limit = 1`. Operacja ta pozwala odczytać elementy o wskazanej wartości klucza partycji [6]. Dopiero po trafieniu obliczany jest pełny skrót h i wykonywane `GetItem` dla pary `CHUNK#<c>/FULL#<h>`; odczyt pojedynczego elementu wymaga wówczas obu składowych klucza [7]. Jeżeli pierwszy odczyt nie zwróci kandydata, pełny skrót jest pomijany na etapie wyszukiwania, ale zostaje uzupełniony przed zapisem zaakceptowanego obrazu. Szczegóły przedstawiono w podrozdziałach 3.3.1 i 3.3.2.

Dzięki temu dokładne duplikaty są odrzucane przed uruchomieniem modelu wektorowego, co ogranicza liczbę wywołań modelu, opóźnienie odpowiedzi oraz koszty obliczeniowe systemu.

2.4. Klasyczna deduplikacja oparta na funkcjach skrótu

W tradycyjnych systemach redundancję danych eliminuje się z użyciem kryptograficznych funkcji skrótu (ang. *hashing algorithms*). Funkcja taka przetwarza strumień wejściowy, generując sygnaturę o stałej długości, która w praktyce służy jako unikalny identyfikator pliku. Zakłada się przy tym, że prawdopodobieństwo kolizji (wygenerowania tego samego skrótu dla dwóch różnych plików) jest pomijalnie małe.

Weryfikacja polega na wyliczeniu skrótu H_{new} dla nowego pliku F_{new} i wyszukaniu go w indeksie systemu. Odczyt rekordu po kluczu należy odróżnić od kosztu obliczenia skrótu: `GetItem` odczytuje wskazany element [7], natomiast SHA-256 przetwarza bloki całego wejścia [22]. Nie oznacza to stałego czasu całej detekcji względem rozmiaru pliku. W rozwiązaniach produkcyjnych najczęściej opiera się to o algorytmy MD5 lub funkcje z rodziny SHA-2.

2.4.1. Algorytm MD5 (*Message-Digest Algorithm 5*)

MD5 kompresuje dane wejściowe w 512-bitowych blokach, zwracając 128-bitowy wynik [23]. Choć ze względu na podatność na kolizje algorytm wycofano z zastosowań ściśle kryptograficznych, jego niska złożoność obliczeniowa sprawia, że nadal bywa wykorzystywany do prostych zadań deduplikacyjnych.

Proces kompresji składa się z 64 kroków (cztery rundy po 16 operacji), wykorzystujących nieliniowe funkcje logiczne. Operują one na 32-bitowych słowach B, C, D według poniższych wzorów [23]:

$$F(B,C,D) = (B \wedge C) \vee (\neg B \wedge D) \tag{2.1}$$

$$G(B,C,D) = (B \wedge D) \vee (C \wedge \neg D) \tag{2.2}$$

$$H(B,C,D) = B \oplus C \oplus D \tag{2.3}$$

$$I(B,C,D) = C \oplus (B \vee \neg D) \tag{2.4}$$

Symbole oznaczają standardowe operacje bitowe: koniunkcję ($\wedge$), alternatywę ($\vee$), alternatywę wykluczającą XOR ($\oplus$) oraz negację logiczną ($\neg$).

2.4.2. Algorytm SHA-256 (*Secure Hash Algorithm 2*)

Bezpieczniejszą alternatywą jest SHA-256, który po przetworzeniu 512-bitowych bloków generuje 256-bitowy wynik [22]. Dwukrotnie dłuższy skrót (2^{256} możliwych wartości) sprawia, że ryzyko kolizji w systemach magazynowania danych jest w praktyce pomijalne.

Algorytm wykorzystuje m.in. rotację w prawo ($ROTR$) i przesunięcie logiczne (SHR). Główne funkcje operujące na bitach to [22]:

$$Ch(E,F,G) = (E \wedge F) \oplus (\neg E \wedge G) \tag{2.5}$$

$$Maj(A,B,C) = (A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C) \tag{2.6}$$

Funkcja Ch (ang. *choose*) wybiera bity z F lub G w zależności od wartości E, natomiast Maj (ang. *majority*) zwraca wartość 1, gdy większość argumentów jest równa 1. Proces

uzupełniają sumy rotacyjne Σ_0 i Σ_1, które zwiększają rozpraszanie (dyfuzję) bitów danych wejściowych [22]:

$$\Sigma_0(A) = ROTR^2(A) \oplus ROTR^{13}(A) \oplus ROTR^{22}(A) \tag{2.7}$$

$$\Sigma_1(E) = ROTR^6(E) \oplus ROTR^{11}(E) \oplus ROTR^{25}(E) \tag{2.8}$$

2.4.3. Porcjowanie zależne od treści (CDC)

Porcjowanie zależne od treści (ang. *Content-Defined Chunking*, CDC) wyznacza granice fragmentów na podstawie zawartości strumienia. W podziale na bloki stałej długości wstawienie danych może przesunąć granice kolejnych fragmentów; CDC ogranicza ten problem, wiążąc podział z lokalną treścią zamiast wyłącznie z pozycją bajtów [24].

Jednym z rozwiązań jest kroczący odcisk Rabina, aktualizowany podczas przesuwania okna po danych. Wynik służy do sprawdzania warunku granicy, natomiast identyfikator wydzielonego fragmentu można obliczyć osobną funkcją kryptograficzną [24]. Takie rozdzielenie zastosowano w projekcie: biblioteka `restic/chunker` realizuje CDC oparte na odcisku Rabina [21], a identyfikatory wyznaczane są funkcją SHA-256 [22]. Wykorzystanie tylko pierwszego fragmentu do filtrowania kandydatów, parametry podziału i odczyt zastępczy opisano w podrozdziale 3.3.1.

2.4.4. Wnioski dla deduplikacji obrazów

Mimo że funkcje skrótu świetnie sprawdzają się przy wykrywaniu identycznych plików binarnych (*bit-to-bit*), nie nadają się do deduplikacji obrazów, które przeszły nawet drobną obróbkę. Główną przeszkodą jest tzw. efekt lawinowy – zmiana nawet jednego bajtu (np. ponowna kompresja JPEG, zmiana rozdzielczości, konwersja z PNG do JPEG czy usunięcie metadanych EXIF) skutkuje kaskadową modyfikacją bitów w wygenerowanym skrócie, dając zupełnie inny wynik. Dla człowieka oba pliki przedstawiają to samo zdjęcie, jednak na poziomie bajtów są to zupełnie różne dane.

Częściowym rozwiązaniem są skróty percepcyjne (ang. *perceptual hashing*, np. pHash), które wyliczają sygnaturę na podstawie zawartości wizualnej obrazu, a nie jego bajtów [25]. Radzą sobie one ze zmianą rozdzielczości czy kompresją JPEG, jednak w badaniach Zaunera żaden z czterech porównanych skrótów nie był odporny na obrót obrazu o 5 stopni ani na jego odbicie lustrzane [25]. Z tego powodu ani algorytmy takie jak SHA-256, ani proste skróty percepcyjne nie pozwalają niezawodnie powiązać ze sobą dwóch wersji tego samego obrazu, co wymusza zastosowanie mechanizmów analizy semantycznej.

2.5. Reprezentacja semantyczna i wektoryzacja obrazów (Embeddings)

Jak pokazano w poprzednim podrozdziale, klasyczne metody weryfikacji oparte na funkcjach skrótu są wrażliwe na najmniejsze zmiany w strukturze pliku. Z tego powodu w nowoczesnych systemach wizji komputerowej (ang. *Computer Vision*) odchodzi się od porównywania obrazów piksel po pikselu na rzecz analizy semantycznej, która skupia się na tym, co obraz faktycznie przedstawia.

2.5.1. Konwolucyjne sieci neuronowe i przestrzeń wielowymiarowa

Podstawą większości współczesnych modeli wizyjnych są konwolucyjne sieci neuronowe (ang. *Convolutional Neural Networks*, CNN), które przetwarzają obraz za pomocą niewielkich filtrów przesuwanych po jego powierzchni [26]. Kolejne warstwy sieci wykrywają coraz bardziej złożone wzorce – od krawędzi i tekstur w warstwach początkowych, aż po kształty i całe obiekty w warstwach głębszych [26].

Wynikiem przetworzenia obrazu jest gęsty wektor (ang. *dense vector*) – tablica liczb rzeczywistych, która opisuje zawartość obrazu jako punkt w wielowymiarowej przestrzeni. Popularna architektura ResNet-50, wykorzystująca połączenia rezydualne do trenowania bardzo głębokich sieci [27], zwraca na wyjściu ostatniej warstwy konwolucyjnej mapę cech o 2048 kanałach. W modelach przeznaczonych do wykrywania kopii obrazów, takich jak SSCD, cechy te są następnie rzutowane na przestrzeń 512-wymiarową [28], [29].

Mapa cech ma jednak postać trójwymiarowego tensora, dlatego przed porównaniem musi zostać zagregowana do pojedynczego wektora. W najprostszym ujęciu jest to uśrednianie wartości każdego kanału (ang. *Average Pooling*), jednak w zadaniach wyszukiwania obrazów lepsze wyniki daje uogólnione uśrednianie potęgowe (ang. *Generalized Mean Pooling*, GeM) [30]:

$$f_k = \left(\frac{1}{|\mathcal{X}_k|} \sum_{x \in \mathcal{X}_k} x^p \right)^{\frac{1}{p}}$$

gdzie $\mathcal{X}_k$ to zbiór aktywacji k-tego kanału mapy cech, a p to parametr potęgi. Dla $p = 1$ otrzymuje się zwykłe uśrednianie, natomiast dla $p \to \infty$ operacja sprowadza się do wyboru wartości maksymalnej (ang. *Max Pooling*) [30]. Model SSCD wykorzystuje wartość $p = 3$ [28].

2.5.2. Matematyczny model uczenia samonadzorowanego (SimCLR i SSCD)

Proces generowania wektorów $\mathbf{z}$ wymaga uprzedniego zebrania rozległego zbioru treningowego, na przykład zbioru DISC21, który zawiera milion obrazów treningowych i milion obrazów referencyjnych [28], [31]. W odróżnieniu od klasycznej klasyfikacji obrazów, uczenie samonadzorowane (ang. *self-supervised learning*) nie wymaga ręcznie przygotowanych etykiet – sieć uczy się wyłącznie na podstawie samych obrazów [32].

W każdym kroku treningu algorytm (np. SimCLR) pobiera partię (ang. *batch*) N obrazów i dla każdego z nich tworzy dwie losowo zmodyfikowane wersje (ang. *augmentations*), np. przycięte, rozmyte lub o zmienionych kolorach [32]. Daje to łącznie $2N$ wektorów, przy czym dwie wersje tego samego obrazu tworzą parę pozytywną, a wszystkie pozostałe obrazy w partii traktowane są jako przykłady negatywne [32]. W przypadku SSCD zestaw modyfikacji rozszerzono o przekształcenia typowe dla obrazów kopiowanych w sieci: kompresję JPEG, obrót, nałożenie tekstu lub emoji, a także mieszanie dwóch obrazów (techniki MixUp i CutMix) [28].

Dysponując wektorami, sieć oblicza ich wzajemne podobieństwo jako iloczyn skalarny wektorów znormalizowanych, skalowany parametrem temperatury τ (w SSCD $\tau =$

0,05) [28], [32]. Funkcja straty InfoNCE dla pary pozytywnej (i, j) przyjmuje postać [28]:

$$\ell_{i,j} = -\log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_j / \tau)}{\sum_{k \neq i} \exp(\mathbf{z}_i \cdot \mathbf{z}_k / \tau)}$$

W procesie optymalizacji wykorzystującym spadek gradientu (ang. *Stochastic Gradient Descent*), algorytm aktualizuje wagi sieci [32]. Celem jest zbliżenie do siebie wektorów dwóch wersji tego samego obrazu poprzez maksymalizację ich iloczynu skalarnego, przy jednoczesnym oddaleniu od siebie przykładów negatywnych. SSCD uzupełnia tę funkcję straty o regularyzację entropii KoLeo [28], [33], która rozkłada wektory różnych obrazów możliwie równomiernie w przestrzeni:

$$\mathscr{L}_{KoLeo} = -\frac{1}{N} \sum_{i=1}^{N} \log \left(\min_{j \notin \hat{P}_i} \|\mathbf{z}_i - \mathbf{z}_j\| \right)$$

gdzie $\hat{P}_i$ oznacza zbiór wersji tego samego obrazu co $\mathbf{z}_i$. Wartość $\mathscr{L}_{InfoNCE}$ jest średnią z $\ell_{i,j}$ po wszystkich parach pozytywnych, a ostateczna funkcja straty ma postać $\mathscr{L} = \mathscr{L}_{InfoNCE} + \lambda \mathscr{L}_{KoLeo}$, gdzie w standardowej konfiguracji $\lambda = 30$ [28]. Dzięki temu model lepiej odróżnia rzeczywiste kopie od obrazów, które są do siebie jedynie podobne tematycznie. W odróżnieniu od SimCLR, w którym warstwa rzutująca jest po treningu odrzucana [32], SSCD zachowuje ją i wykorzystuje także podczas wyliczania wektorów dla nowych obrazów [28].

2.5.3. Porównywanie obrazów i miara cosinusowa

Modele takie jak SSCD przekształcają każdy obraz w pojedynczy wektor o stałej liczbie wymiarów, niezależnie od jego rozdzielczości czy formatu. Przed wyliczeniem wektora obraz jest skalowany do ustalonego rozmiaru (np. krótszy bok do 288 pikseli) i normalizowany [29]. Dzięki temu pliki takie jak oryginalne zdjęcie produktu zapisane w formacie PNG oraz jego pomniejszona, silnie skompresowana kopia w formacie JPEG dają wektory o tej samej liczbie wymiarów, które można bezpośrednio ze sobą porównać. Ponieważ model był trenowany tak, aby kopie tego samego obrazu leżały blisko siebie w przestrzeni wektorowej [28], wektory obu plików będą do siebie zbliżone.

W celu formalnej weryfikacji stopnia podobieństwa między dwoma wektorami $\mathbf{x}$ oraz $\mathbf{y}$, reprezentującymi odpowiednio nowo przesłany obraz oraz obraz referencyjny z bazy danych, standardowo stosuje się miarę cosinusa kąta zawartego między nimi[34]. Oblicza się ją poprzez podzielenie iloczynu skalarnego wektorów przez iloczyn ich długości (znormalizowany iloczyn skalarny)[34]:

$$cosine(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} = \frac{\sum_{i=1}^{d} x_i y_i}{\sqrt{\sum_{i=1}^{d} x_i^2} \sqrt{\sum_{i=1}^{d} y_i^2}}$$

Dzięki normalizacji różnice w fizycznej długości wektorów są ignorowane, a wynikiem jest wyłącznie ocena ich kierunku[34]. Ponieważ wartości gęstych wektorów mogą być ujemne, miara ta przyjmuje wartości w przedziale od 1 (wektory o zgodnym kierunku), przez 0

(wektory ortogonalne, oznaczające brak powiązania), aż do -1 (wektory o przeciwnych kierunkach)[34].

Z perspektywy wydajności obliczeniowej istotnym mechanizmem jest pre-normalizacja, podczas której tworzone są wektory jednostkowe o długości równej 1[34]. Model SSCD fabrycznie zwraca wektory znormalizowane w przestrzeni L2 [29]. Oznacza to, że mianownik w powyższym wzorze przyjmuje wartość równą jeden. Dla wektorów jednostkowych iloczyn skalarny staje się matematycznie tożsamy z miarą cosinusową[34]. Pozwala to na uproszczenie obliczeń do pojedynczej operacji iloczynu skalarnego ($\mathbf{x} \cdot \mathbf{y}$), co przekłada się bezpośrednio na redukcję czasu odpowiedzi bramy deduplikacyjnej. Według autorów modelu `sscd_disc_mixup` pary obrazów DISC, których podobieństwo cosinusowe przekracza 0,75, są kopiami z precyzją 90% [29]. Wartość ta stanowi punkt wyjścia do doboru progu decyzyjnego w projektowanej bramie, jednak ostateczny próg należy wyznaczyć eksperymentalnie dla danych, na których system będzie pracował.

Podobieństwo i odległość kosinusowa opisują różne wielkości. W rozszerzeniu `pgvector` operator `<=>` zwraca odległość. Z podobieństwem wiąże ją zależność [35]:

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \text{cosine}(\mathbf{x}, \mathbf{y}). \tag{2.9}$$

Większe podobieństwo odpowiada mniejszej odległości; przy niezerowych wektorach przedziałowi podobieństwa $[-1, 1]$ odpowiada przedział odległości $[0, 2]$ [34], [35]. W programie parametr `-threshold` określa maksymalną odległość, domyślnie 0,60. Warunek $d_{\cos} \leq 0{,}60$ jest więc równoważny podobieństwu co najmniej 0,40, a nie 0,60. Jest to przeliczenie z równania (2.9), nie wynik strojenia progu. Przykładowemu warunkowi podobieństwa większego niż 0,75 odpowiada odległość mniejsza niż 0,25; wartości tych dwóch miar nie należy stosować zamiennie. Dobór progu dla danych projektu omówiono w podrozdziale 4.4.

2.6. PostgreSQL z rozszerzeniem pgvector

Rozszerzenie `pgvector` dodaje do PostgreSQL typ `vector(n)` oraz operatory odległości, dzięki czemu reprezentacje obrazów można przechowywać i porównywać w zapytaniach SQL [35]. Domyślnie dostępne jest dokładne wyszukiwanie najbliższych sąsiadów. Indeksy przybliżone, w tym HNSW, pozwalają przyspieszyć wyszukiwanie kosztem możliwości pominięcia części najbliższych wyników [35].

HNSW organizuje wektory w wielowarstwowy graf, a przy wyszukiwaniu według odległości kosinusowej używana jest klasa operatorów `vector_cosine_ops` [35]. W projekcie PostgreSQL pełni rolę magazynu reprezentacji obsługiwanego przez AI Workera. Przepływ decyzji i aktualizacji bazy przedstawiono w podrozdziale 3.4, a sposób wyznaczania wektorów w podrozdziale 4.2.

2.7. Komunikacja między usługami: gRPC i Protocol Buffers

gRPC umożliwia wywoływanie metod zdalnej usługi na podstawie kontraktu opisującego parametry i odpowiedzi. Definicje wiadomości oraz metod zapisane w pliku `.proto` służą do wygenerowania kodu klienta i serwera [36]. Protocol Buffers koduje wiadomości

w formacie binarnym; rekordy zawierają numer pola, typ kodowania i dane, a pole `bytes` przenosi ciąg bajtów poprzedzony długością [37].

W bramie Go i AI Workerze Python zastosowano wspólny kontrakt oraz strumieniowanie po stronie klienta. W tym modelu klient wysyła sekwencję wiadomości, serwer zwraca jedną odpowiedź, a kolejność wiadomości w obrębie wywołania jest zachowana [36]. Umożliwia to przesyłanie obrazu we fragmentach zamiast umieszczania całego pliku w jednej wiadomości. Kontrakt projektu i przebieg wywołania przedstawiono w podrozdziałach 3.2.2 i 3.4.

3. Architektura systemu deduplikacyjnego

Rozdział opisuje podział odpowiedzialności między komponentami, przebieg deduplikacji oraz kolejność operacji na danych. Przedstawiono stan implementacji, w tym ograniczenia jej spójności. Sposób wyznaczania reprezentacji obrazów i oceny jakości omówiono oddzielnie w podrozdziałach 4.1 i 4.3; tutaj uwzględniono jedynie ich miejsce w potoku.

3.1. Założenia projektowe i wymagania

Przy projektowaniu uwzględniono wolumen przechowywanych danych, liczbę operacji na indeksach i koszt analizy obrazów. Powiązanie decyzji technicznych z wykorzystaniem zasobów odpowiada zasadom FinOps, w których optymalizację rozpatruje się w odniesieniu do wartości dostarczanej przez system [38]. Przyjęto następujące wymagania:

- **Detekcja duplikatów binarnych:** rozpoznawanie identycznej zawartości przed analizą obrazu.
- **Detekcja kopii po przekształceniach:** porównywanie obrazów także po kompresji, skalowaniu lub innych modyfikacjach. Do takiego zadania opracowano reprezentację SSCD [28], [29].
- **Wybór przechowywanej wersji:** umożliwienie podmiany podobnego obrazu na podstawie oceny jakości.
- **Ograniczona współbieżność:** kontrolowanie liczby jednocześnie przetwarzanych plików i buforowanych strumieni.
- **Kontrola spójności:** określenie zakresu ochrony przed współbieżnymi zapisami i skutków przerwania operacji obejmującej kilka magazynów.

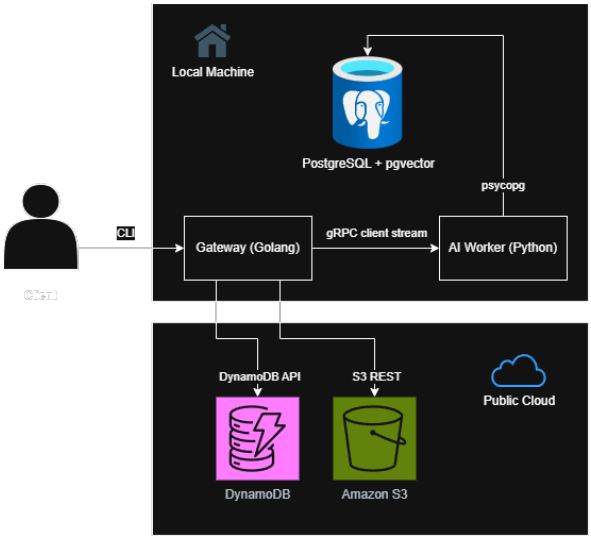
Spełnienia wymagań wydajnościowych nie wywiedziono z samego doboru technologii. Przepustowość, zużycie zasobów i efekty deduplikacji podlegają pomiarom opisanym w rozdziale 5.

3.2. Podział na komponenty i warstwę danych

System obejmuje dwa mikroserwisy połączone interfejsem gRPC. W referencyjnej konfiguracji Docker Compose uruchamiane są one lokalnie, razem z PostgreSQL i LocalStack. LocalStack udostępnia lokalną emulację usług AWS, w tym S3 i DynamoDB, przeznaczoną do prac rozwojowych i testów [39]. Konfiguracja ta nie stanowi wdrożenia usług w rzeczywistym środowisku AWS.

- **Gateway w Go:** obsługuje CLI, odczyt plików, obliczanie skrótów i sterowanie potokiem. Wywołuje AI Workera oraz interfejsy S3 i DynamoDB.
- **AI Worker w Pythonie:** odbiera obrazy, dla których nie potwierdzono duplikatu binarnego. Wyznacza reprezentację i ocenę jakości, wyszukuje odpowiednik w PostgreSQL, a następnie zwraca wynik decyzji.

Warstwę danych podzielono według rodzaju informacji. Interfejs S3 wykorzystano do zapisu obrazów i osobnych obiektów JSON, DynamoDB do przechowywania metadanych



Rysunek 3.1. Architektura Hybrydowa

plików i indeksu par skrótów, a PostgreSQL z rozszerzeniem `pgvector` do przechowywania i przeszukiwania reprezentacji wektorowych. Rozdzielenie tych zadań wykorzystuje odpowiednio model obiektowy, odczyty według klucza i operacje na wektorach [7], [12], [35]. Wektory są zapisywane bezpośrednio przez AI Workera. Nie są zwracane bramie.

Nazwę bazową pliku, uzyskaną przez `filepath.Base`, przyjęto jako identyfikator aplikacyjny. Obraz zapisywany jest pod kluczem `payloads/<file_name>`, a jego opis pod `metadata/<file_name>.json`. Opis JSON jest osobnym obiektem, nie zestawem metadanych użytkownika dołączonych do obiektu obrazu. W DynamoDB zastosowano dwa rodzaje rekordów:

- **Rekord metadanych pliku:** `PK = FILE#<file_name>`, `SK = META`; rekord zawiera m.in. klucz obiektu, rozmiar, wymiary i skróty.
- **Indeks zawartości:** `PK = CHUNK#<c>`, `SK = FULL#<h>`; rekord wiąże skrót pierwszego fragmentu i całego pliku z zapisanym obiektem.

Osobno obsługiwane są powtarzające się nazwy bazowe w jednym przetwarzanym zbiorze. Przed uruchomieniem zadań zachowywane jest pierwsze wystąpienie, a następne zgłaszane są jako błędy wejścia. Takie zachowanie wynika z przyjęcia nazwy bazowej jako identyfikatora, również dla plików pochodzących z różnych katalogów.

3.2.1. Dobór technologii: Go oraz Python

Go wykorzystano do obsługi I/O i organizacji współbieżnego przetwarzania. Goroutines są rozdzielane przez środowisko wykonawcze między wątki systemowe; blokowanie podczas I/O nie musi zatrzymywać pozostałych goroutines [40]. Mechanizm ten nie zastępuje synchronizacji współdzielonego stanu, dlatego równoległość bramy ograniczono jawnie, jak opisano w podrozdziale 3.6.

Python wykorzystano w części analitycznej z PyTorch. Oficjalne repozytorium SSCD udostępnia modele TorchScript i przykłady inferencji w tym środowisku [29]. Rozdzielenie

usług pozwala zmieniać przygotowanie obrazu i inferencję bez przenoszenia tej logiki do kodu obsługującego magazyny danych. W badanej wersji model ładowany jest na CPU.

3.2.2. Komunikacja za pomocą gRPC

Kontrakt zdefiniowano w pliku `proto/dedup.proto`. Na podstawie definicji usługi i wiadomości wygenerowano kod dla obu języków, zgodnie z modelem pracy gRPC [36]. Wykorzystany transport HTTP/2 obsługuje wiele strumieni w jednym połączeniu; nie oznacza to jednak, że obiekt klienta musi utrzymywać zawsze dokładnie jedno połączenie TCP [41], [42].

Listing 1. Fragment kontraktu dedup.proto. Opracowanie własne.

```
service ImageDedupService {
  rpc ProcessImageStream(stream ImageChunk)
      returns (ProcessImageResponse);
}

message ImageChunk {
  string image_key = 1;
  int64 total_size = 2;
  bytes data = 3;
  float threshold = 4;
}

message ProcessImageResponse {
  string status = 1;
  string image_key = 2;
  float quality_score = 3;
  float distance = 4;
  string existing_image_key = 5;
}
```

Metoda `ProcessImageStream` jest wywołaniem ze strumieniowaniem po stronie klienta. Przesyłana jest sekwencja wiadomości `ImageChunk`, po której odbierana jest pojedyncza odpowiedź; gRPC zachowuje kolejność wiadomości w obrębie tego wywołania [36]. Pole `bytes` w Protocol Buffers przenosi dane binarne z zakodowaną długością, bez konieczności zamiany obrazu na tekst [37].

W bramie tworzony jest jeden współdzielony `grpc.ClientConn`. Jest to kanał klienta, a nie zaimplementowana w aplikacji pula połączeń [42]. Adres określa zmienna `AI_WORKER_GRPC_ADDR`, domyślnie `localhost:50051`; w Docker Compose wskazano usługę AI Workera. W tej konfiguracji nie zastosowano TLS ani osobnego limitu czasu dla każdego RPC. Brak jawnego terminu wywołania oznacza, że samo gRPC nie narzuca takiego ograniczenia [43].

Stan transportu odróżniono od pola aplikacyjnego `status`. Błędy RPC przekazywane są kodami gRPC, natomiast pole wiadomości opisuje wynik deduplikacji [44]. AI Worker

jawnie zwraca `INVALID_ARGUMENT`, gdy nie otrzyma klucza obrazu albo danych. Nie zaimplementowano pełnej walidacji zadeklarowanego rozmiaru ani zakresu wartości progu.

3.3. Logiczny potok deduplikacyjny

Przed uruchomieniem potoku odczytywane są rozmiar pliku i konfiguracja obrazu. Niepowodzenie dekodowania, w tym próba odczytu pustego pliku, kończy przetwarzanie przed CDC i RPC. W potoku zastosowano trzy etapy. Najpierw wykonywany jest `ChunkPipeline`, następnie `FullHashPipeline`, a na końcu `SimilarityPipeline`. Każdy zwraca decyzję `continue`, `accept_full` albo `soft_dedup`. Pierwsza decyzja inna niż `continue`, podobnie jak błąd, kończy wykonanie łańcucha.

Etapy jednego pliku wykonywane są sekwencyjnie; współbieżnie przetwarzane są różne pliki. Nie zbudowano oddzielnych procesów roboczych połączonych kanałami dla każdego etapu. Jest to rozróżnienie względem potoków Go, w których równoległe grupy goroutines przekazują dane przez kanały [45]. Pominięcie dalszej analizy wynika z otrzymanej decyzji, nie z założenia, że każdy kolejny etap musi trwać dłużej.

3.3.1. Wstępne wyszukiwanie z wykorzystaniem CDC

Porcjowanie zależne od treści (ang. *Content-Defined Chunking*, CDC) wyznacza granice fragmentów na podstawie danych, a nie wyłącznie pozycji bajtów. Ogranicza przez to skutki przesunięcia granic po wstawieniu lub usunięciu części zawartości, występujące w podziale na bloki o stałej długości [24]. W tym projekcie użyto tylko pierwszego fragmentu jako filtra kandydatów. Nie zaimplementowano przechowywania i deduplikowania wszystkich fragmentów pliku.

Zastosowano bibliotekę `restic/chunker` w wersji `v0.5.0`, wykorzystującą kroczący odcisk Rabina [21]. Odcisk aktualizowany jest dla przesuwanego okna i służy do wyznaczenia granicy, a nie do kryptograficznej identyfikacji zawartości [24]. W użytej wersji okno ma 64 bajty, a domyślny warunek podziału sprawdza 20 najmłodszych bitów odcisku [46]. W aplikacji ustawiono stały wielomian `0x3DA3358B4DC173` stopnia 53.

Domyślne progi biblioteki wynoszą 512 KiB i 8 MiB, a parametr nominalnego średniego rozmiaru odpowiada 1 MiB [21], [46]. Dolny próg ogranicza poszukiwanie granicy, górny wymusza zakończenie fragmentu; koniec pliku pozwala zwrócić również krótszy fragment [46]. Nominalny parametr nie gwarantuje takiej samej średniej w badanym zbiorze danych [24].

Implementacja przewiduje ponadto odczyt zastępczy. Gdy sprawdzenie stopnia wielomianu nie przejdzie albo biblioteka nie zwróci poprawnego, niepustego fragmentu, plik jest przewijany do początku i odczytywane jest do 64 KiB. Nie jest to wówczas podział CDC. Bufor początkowy tej samej wielkości przekazywany do biblioteki nie ogranicza prawidłowego fragmentu CDC do 64 KiB.

Niech $C_1(f)$ oznacza fragment rzeczywiście zwrócony przez `readFirstChunk`: pierwszy fragment CDC albo, w ścieżce zastępczej, prefiks długości $\min(|f|, 64\,\text{KiB})$. Zdefiniowano dwa identyfikatory:

$$c = \mathrm{SHA256}\big(C_1(f)\big), \qquad h = \mathrm{SHA256}(f). \tag{3.1}$$

Definicję funkcji SHA-256 określa standard FIPS 180-4 [22]. Równanie (3.1) jest natomiast oznaczeniem przyjętym w projekcie i nie zakłada jednoczesnego obliczenia obu skrótów. W zwykłej ścieżce CDC plik krótszy niż 512 KiB stanowi cały pierwszy fragment; wtedy $c = h$. W ścieżce zastępczej taka równość wynika z objęcia całego pliku prefiksem tylko dla rozmiaru nieprzekraczającego 64 KiB.

Po wyznaczeniu c wykonywana jest operacja `Query` dla `PK = CHUNK#<c>` z parametrem `Limit = 1`. Limit ogranicza liczbę ocenianych elementów, nie stanowi ogólnej gwarancji uzyskania jednego dopasowania po zastosowaniu filtra [47]. W badanym kodzie nie użyto `FilterExpression`, dlatego obecność zwróconego elementu wystarcza do wskazania kandydata. Wynik zapisywany jest we fladze `ChunkMatched`.

```
c = sha256(readFirstChunk(file))
Query PK = CHUNK#c, Limit = 1
  miss -> skip exact full-hash lookup; similarity
  hit:
    h = sha256(file)
    GetItem PK = CHUNK#c, SK = FULL#h
      hit  -> binary soft-dedup
      miss -> similarity
```

Rysunek 3.2. Dwuetapowe wyszukiwanie duplikatu binarnego. Opracowanie własne na podstawie implementacji.

3.3.2. Weryfikacja pełnego skrótu i warunki kontynuacji

Trafienie dla c nie potwierdza tożsamości plików. Mogą one zawierać taki sam początkowy fragment i różnić się dalszą zawartością. Nie jest to kolizja funkcji skrótu, ponieważ haszowany fragment jest w obu przypadkach identyczny. Po trafieniu plik otwierany jest ponownie, a h obliczane z całej zawartości, łącznie z wcześniej odczytanym początkiem.

Następnie wykonywane jest `GetItem` dla `PK = CHUNK#<c>` i `SK = FULL#<h>`. Operacja odczytuje rekord po pełnym kluczu głównym, obejmującym w tej tabeli klucz partycji i klucz sortowania [7]. Trafienie kończy łańcuch decyzją `soft_dedup`. Nie uruchamia się RPC ani zapisu kolejnej kopii obrazu. Nadal zapisywany jest jednak obiekt JSON z metadanymi przetwarzania.

Niech $\mathscr{I}$ oznacza ustalony zbiór par skrótów (c', h') w indeksie, a $\mathscr{C} = \{c' \mid \exists h' : (c', h') \in \mathscr{I}\}$ zbiór skrótów początkowych fragmentów. Warunek rozpoznania duplikatu w gałęzi binarnej zapisano jako:

$$\mathrm{dup}_{\mathrm{bin}}(f) \iff (c, h) \in \mathscr{I}. \tag{3.2}$$

Jest to formalizacja schematu indeksowania, nie wzór zaczerpnięty z definicji CDC. Obecność pary implikuje obecność c w $\mathscr{C}$, więc pierwsze zapytanie pozwala odroczyć obli-

czenie h, zamiast dostarczać osobnego kryterium tożsamości. Predykatu $\mathrm{dup}_{\mathrm{bin}}$ nie utożsamiono z każdą decyzją `soft_dedup`. Dotyczy ona także statusu `duplicate_rejected` z gałęzi podobieństwa. Zgodność skrótów przyjęto jako kryterium projektu, bez porównywania plików bajt po bajcie i bez zakładania matematycznej niemożliwości kolizji.

Jeżeli odczyty zakończą się poprawnie, kontynuacja do analizy podobieństwa odpowiada warunkowi:

$$\mathrm{continue}_{\mathrm{bin}}(f) \iff (c \notin \mathscr{C}) \vee \big(c \in \mathscr{C} \wedge (c,h) \notin \mathscr{I}\big). \tag{3.3}$$

W wykonaniu stosowana jest ocena warunkowa. Brak kandydata pozwala przejść do analizy podobieństwa bez obliczania h na potrzeby wyszukania duplikatu. Nie oznacza to pominięcia pełnego skrótu przez cały proces: jeśli obraz zostanie zaakceptowany do zapisu, brakujące h jest obliczane w `uploadImage` przed przesłaniem danych i utworzeniem wpisu indeksu.

```
no CHUNK#c:
  similarity without full hash in lookup stage
  accepted -> compute missing full hash before upload

CHUNK#c exists, no matching FULL#h:
  similarity with full hash already available
```

Rysunek 3.3. Dwie ścieżki kontynuacji po niepotwierdzeniu duplikatu. Opracowanie własne na podstawie implementacji.

W kodzie nie ustawiono `ConsistentRead`. W usłudze DynamoDB oznacza to domyślnie spójność ostateczną: odczyt może nie uwzględniać niedawnego zapisu [48]. Równania (3.2) i (3.3) opisują regułę dla ustalonego stanu indeksu, nie atomowy obraz dwóch kolejnych wywołań. Brak trafienia oznacza zatem niepotwierdzenie duplikatu w odczytanym stanie, a nie bezwarunkową gwarancję unikalności.

3.4. Integracja z modułem analitycznym i obsługa wyników

3.4.1. Przesłanie i buforowanie obrazu

Po niepotwierdzeniu duplikatu binarnego plik otwierany jest ponownie i przesyłany od początku metodą `ProcessImageStream` usługi `dedup.v1.ImageDedupService`. Podział transportowy jest niezależny od CDC. W kliencie zastosowano bufor o pojemności $B = 2\,\mathrm{MiB} = 2 \cdot 2^{20}$ bajtów. Każdy dodatni wynik wywołania `Read` wysyłany jest w osobnej wiadomości.

Interfejs `io.Reader` dopuszcza odczyt krótszy od pojemności bufora, także przed końcem danych [49]. Dla poprawnie przesłanego niepustego pliku, bez wcześniejszego zakończenia strumienia, oznaczono przez b_i liczbę bajtów obrazu w wiadomości i, a przez s rozmiar pliku. Wówczas:

$$\sum_{i=1}^{n} b_i = s, \qquad 0 < b_i \le B \quad \Longrightarrow \quad n \ge \left\lceil \frac{s}{B} \right\rceil. \tag{3.4}$$

Równość zachodzi, gdy wszystkie wiadomości poza ostatnią zawierają pełne B bajtów danych. Nie jest to gwarancja obecnej implementacji, ponieważ nie zastosowano dopełniania krótkich odczytów przez `io.ReadFull` [49]. Równanie wynika z przyjętej pojemności bufora, a nie z właściwości algorytmu CDC.

Pierwsza wiadomość zawiera dane obrazu oraz pola `image_key`, `total_size` i `threshold`. W kolejnych wiadomościach wypełniane jest tylko `data`. Limit odbioru i wysyłania po stronie AI Workera ustawiono na 8 MiB na wiadomość. Limit wiadomości gRPC nie ogranicza łącznego rozmiaru strumienia, a do danych obrazu dochodzi narzut kodowania pól Protocol Buffers [37], [50].

Po stronie odbiorcy użyto `SpooledTemporaryFile` z progiem 20 MiB. Po jego przekroczeniu bufor przechodzi z pamięci operacyjnej do pliku tymczasowego [51]. AI Worker zbiera cały strumień, a dopiero potem uruchamia ocenę jakości i inferencję. Strumieniowy transport nie oznacza więc inferencji na niekompletnym obrazie. Próg bufora nie jest też limitem całkowitej pamięci procesu, ponieważ poza nim przechowywane są model i zdekodowane dane obrazu.

Jeżeli `Send` zwróci `io.EOF`, klient kończy wysyłanie i wywołuje `CloseAndRecv`, aby odebrać wynik lub końcowy status RPC. Jest to zgodne z możliwością zakończenia wywołania przez serwer przed odebraniem wszystkich wiadomości klienta [36]. Inne błędy wysyłania przerywają przetwarzanie.

3.4.2. Decyzja AI Workera

Pole `threshold` przenosi maksymalną dopuszczalną odległość kosinusową, domyślnie 0,60, a nie minimalne podobieństwo. W zapytaniu do `pgvector` użyto operatora `<=>`; odpowiada on odległości kosinusowej, podczas gdy podobieństwo wynosi jeden minus ta odległość [35]. Definicję reprezentacji i dobór progu przedstawiono w podrozdziałach 4.2 i 4.4.

AI Worker wykonuje wyszukanie i ewentualną aktualizację PostgreSQL przed zwróceniem odpowiedzi RPC. Odpowiedź zawiera identyfikatory, ocenę jakości i odległość, ale nie reprezentację wektorową. W bramie status mapowany jest na dalsze działanie:

- `inserted`**:** przyjmowany jest zapis obrazu. W PostgreSQL użyto operacji `INSERT ... ON CONFLICT`, więc status nie gwarantuje utworzenia nowego wiersza przy ponownym użyciu tego samego `image_key`.
- `duplicate_rejected`**:** zwracana jest decyzja `soft_dedup`. Pomijany jest zapis obrazu i wpisów DynamoDB, ale zapisywany jest obiekt JSON.
- `replaced`**:** przyjmowany jest zapis obrazu i ustawiana flaga `ReplaceExisting`. Rekord PostgreSQL aktualizowany jest w miejscu przez `UPDATE`, bez poprzedzającego `DELETE`.

3.4.3. Kolejność zapisu i wycofanie poprzedniej wersji

Po poprawnym zakończeniu łańcucha serializowany i zapisywany jest JSON. Przy `soft_dedup` proces kończy się na tym etapie. Dla zaakceptowanego obrazu najpierw uzupełniany jest brakujący pełny skrót. Następnie obraz zapisywany jest pod kluczem `payloads/<file_name>`. Na końcu zapisywane są rekord metadanych pliku `FILE#<file_name>/META` i para `CHUNK#<c>/FULL#<h>`. Są to oddzielne operacje, a nie jedna transakcja.

Funkcja `retireReplaced` uruchamiana jest dopiero po poprawnym zapisaniu nowego obrazu i obu wpisów indeksu. Jeżeli `existing_image_key` różni się od nowej nazwy, odczytywany jest stary rekord metadanych pliku, a następnie usuwane są: poprzedni obraz, jego JSON, rekord metadanych oraz, jeśli odczytano potrzebne skróty, rekord zawartości. Przy identycznym kluczu osobne usuwanie jest pomijane. Żadna z tych ścieżek nie usuwa lokalnego pliku źródłowego.

Skrypt inicjalizacyjny nie włącza wersjonowania zasobnika. Nie należy jednak utożsamiać tego z gwarancją konfiguracji każdego środowiska, do którego zostanie podłączona brama. Przy włączonym wersjonowaniu S3 ponowny zapis pod tym samym kluczem tworzy nową wersję, a usunięcie bez identyfikatora wersji dodaje znacznik usunięcia zamiast kasować wcześniejsze wersje [52].

Dla statusu `replaced` rozmiar poprzedniego obrazu odczytywany jest przez `HeadObject`. Niech k_{old} i k_{new} oznaczają klucze, a s_{old} i s_{new} rozmiary w bajtach. Licznik projektu ma postać:

$$\texttt{deleted_bytes} = \begin{cases} 0, & \text{gdy odczyt rozmiaru nie powiedzie się,} \\ \max(0, s_{\text{old}} - s_{\text{new}}), & \text{gdy } k_{\text{old}} = k_{\text{new}}, \\ s_{\text{old}}, & \text{gdy } k_{\text{old}} \neq k_{\text{new}}. \end{cases} \tag{3.5}$$

Dwie ostatnie gałęzie zakładają poprawny odczyt rozmiaru. Przy tym samym kluczu liczona jest dodatnia różnica rozmiarów, przy różnych kluczach cały rozmiar poprzedniego obiektu. Nie jest to zatem jednolita miara oszczędności netto. Ponadto S3 rozlicza zachowane wersje jako pełne obiekty, nie różnice między wersjami [52]. Interpretację kosztową pozostawiono analizie w rozdziale 5.

3.5. Miejsce oceny jakości w potoku

Ocenę podobieństwa oddzielono od wyboru przechowywanej wersji. Pierwsza wskazuje kandydata, druga rozstrzyga decyzję obsługiwaną przez AI Workera. W bramie nie występuje osobny etap `InfoGain`; klasa `SimilarityPipeline` odbiera gotowy status i ustawia decyzję zapisu. Plik `decision.go` definiuje możliwe decyzje potoku, nie algorytm oceny jakości.

Wynik oceny przekazywany jest w polu `quality_score`. Pole `existing_image_key` wskazuje obiekt podlegający ewentualnemu wycofaniu. Definicję wskaźnika, kierunek porównania i obsługę remisu przedstawiono w podrozdziale 4.3.

3.6. Współbieżność i zakres gwarancji spójności

3.6.1. Limity i anulowanie pracy

Do ograniczenia współbieżności użyto pakietu `errgroup`. Grupę tworzono przez `WithContext`, a limit ustawiano metodą `SetLimit`. Parametr `-workers` ma domyślną wartość 10; przyjęto co najmniej jedno zadanie. `SetLimit` ogranicza liczbę aktywnych goroutines grupy, a dodanie następnej może oczekiwać na zwolnienie miejsca [53]. Nie jest to stała pula połączona kolejką aplikacyjną.

Wewnętrznie `errgroup` może anulować kontekst po zwróceniu pierwszego błędu przez zadanie [53]. W tej aplikacji błędy pojedynczych obrazów są jednak rejestrowane, a funkcja zadania zwraca `nil`. Niepowodzenie jednego pliku nie przerywa więc automatycznie całego zbioru; po zakończeniu zliczane są nieudane operacje. Kontekst obsługuje natomiast przerwanie przez `SIGINT` lub `SIGTERM`.

AI Worker wykorzystuje `ThreadPoolExecutor` z limitem czterech wątków obsługi gRPC. Przed rozpoczęciem przyjmowania wywołań inicjalizowany jest schemat bazy i ładowany model. Limit ten jest niezależny od liczby zadań bramy i nie stanowi gwarancji czterokrotnego przyspieszenia inferencji. Próg buforowania 20 MiB dotyczy pojedynczego strumienia, nie sumy danych wszystkich żądań.

3.6.2. Transakcja decyzji i operacje między magazynami

W `Repository.resolve` otwierana jest transakcja PostgreSQL. Następnie pobierana jest blokada `pg_advisory_xact_lock` ze stałym identyfikatorem 74821001. Transakcyjne blokady doradcze są zwalniane na końcu transakcji i wymagają współpracy aplikacji używających tej samej konwencji [54]. Wspólny identyfikator szereguje całe sekcje rozstrzygania decyzji w tej samej bazie, również dla różnych obrazów. Blokada nie obejmuje wcześniejszego obliczania reprezentacji i jakości.

Przy odczycie jakości istniejącego obrazu stosowane jest także `SELECT ... FOR UPDATE`. Taka blokada chroni wskazany wiersz przed konkurencyjną modyfikacją do końca transakcji [54]. Wyjście z kontekstu transakcji Psycopg kończy się zatwierdzeniem albo wycofaniem zmian zależnie od wystąpienia wyjątku [55]. Zatwierdzenie następuje przed zwróceniem odpowiedzi do bramy.

Ochrona PostgreSQL nie obejmuje późniejszych zapisów JSON, obrazu i indeksów DynamoDB. W kodzie nie zastosowano wspólnej transakcji, zapisów warunkowych DynamoDB ani procedury kompensacji po błędzie. Przerwanie po aktualizacji PostgreSQL może zatem pozostawić rekord wektorowy bez odpowiadającego mu nowego obrazu; przerwanie kolejnych zapisów może pozostawić niepełny indeks. Nie można na tej podstawie zadeklarować atomowej podmiany we wszystkich magazynach.

Pozostaje też ograniczenie podmiany pod tą samą nazwą. `retireReplaced` pomija wtedy czyszczenie, mimo że nowy zapis może dotyczyć innych skrótów. Stary rekord `CHUNK#<c>/FULL#<h>` może nadal wskazywać nadpisany obiekt. Z tego względu poprawność kolejnego trafienia binarnego zależy również od aktualności indeksu, a nie wyłącznie

od zgodności skrótów. Ograniczenie to wymaga usunięcia w implementacji lub uwzględnienia w ocenie wyników.

4. Moduł sztucznej inteligencji i analiza semantyczna obrazów

4.1. Ekstrakcja cech i model SSCD (Self-Supervised Descriptor)

4.2. Reprezentacja wektorowa i przeszukiwanie przestrzeni wielowymiarowej

4.3. Obiektywna ocena jakości obrazu (InfoGain) i logika decyzyjna

4.4. Ewaluacja i strojenie progów podobieństwa (Threshold Tuning)

5. Ewaluacja, strojenie progów i analiza efektywności (FinOps)

5.1. Metodologia eksperymentu i przygotowanie danych

5.2. Strojenie progów podobieństwa (Threshold Tuning)

5.3. Wydajność potoku deduplikacyjnego (Pipeline Performance)

5.4. Analiza finansowa i optymalizacja kosztów chmurowych (FinOps)

6. Summatio

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Donec odio elit, dictum in, hendrerit sit amet, egestas sed, leo. Praesent feugiat sapien aliquet odio. Integer vitae justo. Aliquam vestibulum fringilla lorem. Sed neque lectus, consectetuer at, consectetuer sed, eleifend ac, lectus. Nulla facilisi. Pellentesque eget lectus. Proin eu metus. Sed porttitor. In hac habitasse platea dictumst. Suspendisse eu lectus. Ut mi mi, lacinia sit amet, placerat et, mollis vitae, dui. Sed ante tellus, tristique ut, iaculis eu, malesuada ac, dui. Mauris nibh leo, facilisis non, adipiscing quis, ultrices a, dui.

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetuer a, feugiat vitae, porttitor eu, libero. Suspendisse sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices

posuere cubilia Curae; Pellentesque sit amet pede ac sem eleifend consectetuer. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius, eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.

Bibliografia

[1] M. Goossens, F. Mittelbach i A. Samarin, *The LaTeX Companion*. Reading, Massachusetts: Addison-Wesley, 1993.

[2] Wikimedia Commons. „File:Amazon Web Services Logo.svg", dostęp 25 października 2026. URL: https://commons.wikimedia.org/wiki/File:Amazon_Web_Services_Logo.svg

[3] Amazon Web Services. „What is AWS?", dostęp 25 października 2026. URL: https://aws.amazon.com/what-is-aws/

[4] Amazon Web Services. „AWS Pricing", dostęp 25 października 2026. URL: https://aws.amazon.com/pricing/

[5] Amazon Web Services. „Global Infrastructure – AWS Regions and Availability Zones", dostęp 25 października 2026. URL: https://aws.amazon.com/about-aws/global-infrastructure/

[6] Amazon Web Services. „Query: Amazon DynamoDB API Reference", dostęp 25 października 2026. URL: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_Query.html

[7] Amazon Web Services. „GetItem". Amazon DynamoDB API Reference; dokumentacja aktualizowana na bieżąco, dostęp 25 października 2026. URL: https://docs.aws.amazon.com/amazondynamodb/latest/APIReference/API_GetItem.html

[8] Amazon Web Services. „AWS SDKs and Tools – Reference Guide", dostęp 25 października 2026. URL: https://docs.aws.amazon.com/sdkref/latest/guide/overview.html

[9] Amazon Web Services. „How AWS Pricing Works", dostęp 25 października 2026. URL: https://aws.amazon.com/pricing/how-aws-pricing-works/

[10] Amazon Web Services. „AWS Architecture Icons", dostęp 14 października 2026. URL: https://aws.amazon.com/architecture/icons/

[11] Amazon Web Services. „Amazon S3 – Chmurowy Magazyn Obiektowy", dostęp 13 września 2026. URL: https://aws.amazon.com/pm/serv-s3/

[12] Amazon Web Services. „What is Amazon S3? – Amazon Simple Storage Service User Guide", dostęp 13 września 2026. URL: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html

[13] Amazon Web Services. „Namespaces for general purpose buckets – Amazon S3 User Guide", dostęp 14 października 2026. URL: https://docs.aws.amazon.com/AmazonS3/latest/userguide/gpbucketnamespaces.html

[14] Amazon Web Services. „Amazon S3 increases the maximum object size to 50 TB", dostęp 14 października 2026. URL: https://aws.amazon.com/about-aws/whats-new/2025/12/amazon-s3-maximum-object-size-50-tb/

[15] Amazon Web Services. „Checking object integrity for data uploads in Amazon S3 – Amazon S3 User Guide", dostęp 14 października 2026. URL: https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity-upload.html

[16] Amazon Web Services. „Working with object metadata: Amazon S3 User Guide”, dostęp 25 października 2026. URL: https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingMetadata.html

[17] Amazon Web Services. „PutObject: Amazon S3 API Reference”, dostęp 25 października 2026. URL: https://docs.aws.amazon.com/AmazonS3/latest/API/API_PutObject.html

[18] Amazon Web Services. „Amazon DynamoDB Documentation”, dostęp 14 października 2026. URL: https://aws.amazon.com/documentation-overview/dynamodb/

[19] Amazon Web Services. „What is a Key Value Database? – AWS”, dostęp 14 października 2026. URL: https://aws.amazon.com/nosql/key-value/

[20] Amazon Web Services. „Core components of Amazon DynamoDB”. Klucz prosty i klucz złożony, dostęp 25 października 2026. URL: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.CoreComponents.html

[21] The restic Authors. „chunker: package github.com/restic/chunker”. wersja 0.5.0. Dokumentacja wersji v0.5.0 wskazanej w go.mod badanego projektu, dostęp 25 października 2026. URL: https://pkg.go.dev/github.com/restic/chunker@v0.5.0

[22] National Institute of Standards and Technology, *FIPS 180-4: Secure Hash Standard (SHS)*, 2015. dostęp 25 października 2026. URL: https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[23] R. L. Rivest. „RFC 1321: The MD5 Message-Digest Algorithm”, dostęp 18 października 2026. URL: https://www.rfc-editor.org/info/rfc1321/

[24] M. Gregoriadis, L. Balduf, B. Scheuermann i J. Pouwelse, „A Thorough Investigation of Content-Defined Chunking Algorithms for Data Deduplication”, *arXiv*, 28 września 2024, Preprint, wersja 3. DOI: 10.48550/arXiv.2409.06066 arXiv: 2409.06066 [cs.DC]. dostęp 25 października 2026. URL: https://arxiv.org/html/2409.06066v3

[25] C. Zauner, „Implementation and Benchmarking of Perceptual Image Hash Functions”, praca magisterska, Upper Austria University of Applied Sciences, Hagenberg Campus, 2010. URL: https://www.phash.org/docs/pubs/thesis_zauner.pdf

[26] I. Goodfellow, Y. Bengio i A. Courville, *Deep Learning*. MIT Press, 2016. URL: https://www.deeplearningbook.org

[27] K. He, X. Zhang, S. Ren i J. Sun, „Deep Residual Learning for Image Recognition”, w: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, s. 770–778. URL: https://arxiv.org/abs/1512.03385

[28] E. Pizzi, S. D. Roy, S. N. Ravindra, P. Goyal i M. Douze, „A Self-Supervised Descriptor for Image Copy Detection”, w: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, s. 14 532–14 542. URL: https://arxiv.org/abs/2202.10261

[29] Meta AI Research. „SSCD: A Self-Supervised Descriptor for Image Copy Detection – repozytorium kodu i modeli", dostęp 25 października 2026. URL: https://github.com/facebookresearch/sscd-copy-detection

[30] F. Radenović, G. Tolias i O. Chum, „Fine-Tuning CNN Image Retrieval with No Human Annotation", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, t. 41, nr 7, s. 1655–1668, 2019. URL: https://arxiv.org/abs/1711.02512

[31] M. Douze i in., „The 2021 Image Similarity Dataset and Challenge", *arXiv preprint arXiv:2106.09672*, 2021. URL: https://arxiv.org/abs/2106.09672

[32] T. Chen, S. Kornblith, M. Norouzi i G. Hinton, „A Simple Framework for Contrastive Learning of Visual Representations", w: *Proceedings of the 37th International Conference on Machine Learning (ICML)*, seria PMLR, t. 119, 2020, s. 1597–1607. URL: https://arxiv.org/abs/2002.05709

[33] A. Sablayrolles, M. Douze, C. Schmid i H. Jégou, „Spreading Vectors for Similarity Search", w: *International Conference on Learning Representations (ICLR)*, 2019. URL: https://arxiv.org/abs/1806.03198

[34] D. Jurafsky i J. H. Martin, *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*, 3rd. 2026, Online manuscript released August 19, 2026. URL: https://web.stanford.edu/~jurafsky/slp3/

[35] The pgvector Authors. „pgvector: Open-source vector similarity search for Postgres". Dokumentacja projektu: miary odległości i indeksy, dostęp 25 października 2026. URL: https://github.com/pgvector/pgvector

[36] The gRPC Authors. „Core concepts, architecture and lifecycle", dostęp 25 października 2026. URL: https://grpc.io/docs/what-is-grpc/core-concepts/

[37] Google. „Encoding: Protocol Buffers Documentation", dostęp 25 października 2026. URL: https://protobuf.dev/programming-guides/encoding/

[38] FinOps Foundation. „FinOps Principles", dostęp 25 października 2026. URL: https://www.finops.org/framework/principles/

[39] LocalStack. „Getting Started with LocalStack for AWS: Overview", dostęp 25 października 2026. URL: https://docs.localstack.cloud/aws/getting-started/

[40] The Go Authors. „Effective Go". Część Concurrency: goroutines, kanały i synchronizacja, dostęp 25 października 2026. URL: https://go.dev/doc/effective_go

[41] M. Thomson i C. Benfield, „HTTP/2", Internet Engineering Task Force, Request for Comments 9113, czerwiec 2022. dostęp 25 października 2026. URL: https://www.rfc-editor.org/rfc/rfc9113.html

[42] The gRPC Authors. „grpc: package google.golang.org/grpc". NewClient i ClientConn, dostęp 25 października 2026. URL: https://pkg.go.dev/google.golang.org/grpc

[43] The gRPC Authors. „Deadlines", dostęp 25 października 2026. URL: https://grpc.io/docs/guides/deadlines/

[44] The gRPC Authors. „Status Codes", dostęp 25 października 2026. URL: https://grpc.io/docs/guides/status-codes/

[45] S. Ajmani. „Go Concurrency Patterns: Pipelines and cancellation”, dostęp 25 października 2026. URL: https://go.dev/blog/pipelines

[46] The restic Authors. „restic/chunker: chunker.go, v0.5.0”. wersja 0.5.0. Kod źródłowy: warunek podziału, ograniczenia rozmiaru i obsługa końca strumienia, dostęp 25 października 2026. URL: https://github.com/restic/chunker/blob/v0.5.0/chunker.go

[47] Amazon Web Services. „Other aspects of working with the Query operation in DynamoDB”. Amazon DynamoDB Developer Guide; dokumentacja aktualizowana na bieżąco, dostęp 25 października 2026. URL: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Query.Other.html

[48] Amazon Web Services. „DynamoDB read consistency”, dostęp 25 października 2026. URL: https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

[49] The Go Authors. „io: Go standard library”. Kontrakt Reader.Read i funkcja ReadFull, dostęp 25 października 2026. URL: https://pkg.go.dev/io

[50] The gRPC Authors. „gRPC Core: Grpc_arg_keys”. Konfiguracja limitu rozmiaru odbieranej wiadomości, dostęp 25 października 2026. URL: https://grpc.github.io/grpc/core/group__grpc__arg__keys.html

[51] Python Software Foundation. „tempfile: Generate temporary files and directories”. Dokumentacja klasy SpooledTemporaryFile, dostęp 25 października 2026. URL: https://docs.python.org/3/library/tempfile.html

[52] Amazon Web Services. „How S3 Versioning works”, dostęp 25 października 2026. URL: https://docs.aws.amazon.com/AmazonS3/latest/userguide/versioning-workflows.html

[53] The Go Authors. „errgroup: package golang.org/x/sync/errgroup”. Dokumentacja WithContext, Go i SetLimit, dostęp 25 października 2026. URL: https://pkg.go.dev/golang.org/x/sync/errgroup

[54] The PostgreSQL Global Development Group. „PostgreSQL 16 Documentation: Explicit Locking”. Blokady doradcze transakcji i blokady wierszy, dostęp 25 października 2026. URL: https://www.postgresql.org/docs/16/explicit-locking.html

[55] The Psycopg Team. „Psycopg 3 Documentation: Transactions management”, dostęp 25 października 2026. URL: https://www.psycopg.org/psycopg3/docs/basic/transactions.html

Wykaz symboli i skrótów

EiTI – Wydział Elektroniki i Technik Informacyjnych
PW – Politechnika Warszawska
WEIRD – ang. *Western, Educated, Industrialized, Rich and Democratic*

Spis rysunków

1.1 Tradycyjne godło Politechniki Warszawskiej 7
2.1 Logo Amazon Web Services. Źródło: Wikimedia Commons [2]. 9
2.2 Ikona usługi Amazon S3. Źródło: AWS Architecture Icons [10]. 9
2.3 Ikona usługi Amazon DynamoDB. Źródło: AWS Architecture Icons [10]. 11
3.1 Architektura Hybrydowa 19
3.2 Dwuetapowe wyszukiwanie duplikatu binarnego. Opracowanie własne na podstawie implementacji. 22
3.3 Dwie ścieżki kontynuacji po niepotwierdzeniu duplikatu. Opracowanie własne na podstawie implementacji. 23

Spis tabel

1.1 Przykładowa tabela. 8

Spis załączników

1. Nazwa załącznika 1 38
2. Nazwa załącznika 2 40

Załącznik 1. Nazwa załącznika 1

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa. Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel



Rysunek 1.1. Obrazek w załączniku.

justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Załącznik 2. Nazwa załącznika 2

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris. Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at,

Tabela 2.1. Tabela w załączniku.

Kolumna 1	Kolumna 2	Liczba
cell1	cell2	60
	Suma:	123,45

tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.